



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER II, SESSION 2024/2025

BACHELOR OF COMPUTER SCIENCE (BIOINFORMATICS)

SECJ2154 OBJECT-ORIENTED PROGRAMMING - SECTION 01

PROJECT

CAMPUS FACILITIES BOOKING SYSTEM

GROUP NAME: DATA VOYAGERS

GROUP MEMBERS	MATRIC NO
TAN ZHAO HONG (LEADER)	A23CS0188
CHIN PEI WEN	A23CS0065
KOO XUAN	A23CS0300
NAZATUL NADHIRAH BINTI SABTU	A23CS0144

LECTURER'S NAME : DR. NORSHAM BINTI IDRIS

SUBMISSION DATE : 26 JUNE 2025

TABLE OF CONTENT

TABLE OF CONTENT	1
1.0 Introduction.....	2
1.1 Objective of the system.....	3
1.2 Purposes or significance of the system.....	3
2.0 Use Case Diagram.....	4
3.0 Class Diagram.....	5
4.0 A complete Java program.....	6
4.1 Bookable.java.....	6
4.2 Facility.java.....	6
4.3 FacilityManager.java.....	7
4.4 Gymnasium.java.....	8
4.5 SwimmingPool.java.....	9
4.6 SportsComplex.java.....	10
4.7 Clinic.java.....	11
4.8 Notifiable.java.....	11
4.9 Person.java.....	12
4.10 User.java.....	13
4.11 UserManager.java.....	14
4.12 WaitlistEntry.java.....	14
4.13 Booking.java.....	15
4.14 BookingManager.java.....	16
4.15 BookingException.java.....	17
4.16 FacilityNotAvailableException.java.....	17
4.17 InvalidUserException.java.....	18
4.18 BookingNotFoundException.java.....	18
4.19 CampusFacilityBookingSystem.java (main).....	19
5.0 Task Allocation for each member of the group.....	25
6.0 Explanation of the object-oriented concepts applied.....	26
6.1 Encapsulation and Data Hiding.....	26
6.2 Class Relationships.....	27
6.3 Polymorphism.....	28
6.4 Interfaces and Implementation.....	29
6.5 Exception Handling.....	30
7.0 Conclusion.....	31

1.0 Introduction

In a modern campus environment, the efficient and organized management of campus resources plays a crucial role in supporting both academic and extracurricular activities. Facilities such as gymnasiums, swimming pools, sports complexes and health clinics are frequently used by students, faculty and staff. Without a proper system, booking and managing these facilities can lead to conflicts, double bookings, underutilization or unnecessary delays, which reduces the overall user experience and productivity.

The Campus Facility Booking System is a Java-based object-oriented application designed to automate and manage the booking process for various university facilities. It provides a streamlined platform for users to view, request and manage facility booking while allowing administrators to monitor and control availability, resolve booking conflicts and maintain waitlists efficiently.

This system demonstrates the application of fundamental software engineering and object-oriented programming principles including encapsulation, inheritance, abstraction and polymorphism. It uses Java classes and interfaces to create a modular and extensible system that separates concerns between users, bookings, facilities and system administrators.

By using this system, the university can reduce manual processing, eliminate paper-based bookings, ensure fair access to facilities and maintain accurate records of all activities. It not only simplifies the booking experience for end-users but also provides administrators with tools to manage limited resources more efficiently.

1.1 Objective of the system

- To develop a user-friendly booking system for campus facilities that supports multiple users and various facility types
- To automate the booking process, reducing manual errors and administrative overhead
- To implement a waitlist mechanism for fully booked facilities
- To integrate exception handling for scenarios such as double booking, unavailable facilities
- To demonstrate proper use of object-oriented programming concepts in a practical application

1.2 Purposes or significance of the system

- Simplifies and automates the campus facility booking process.
- Reduces manual errors and administrative workload.
- Ensures fair and transparent access to limited resources.
- Demonstrates object-oriented programming in a real-world context.
- Acts as a practical learning tool for students in software development.

2.0 Use Case Diagram

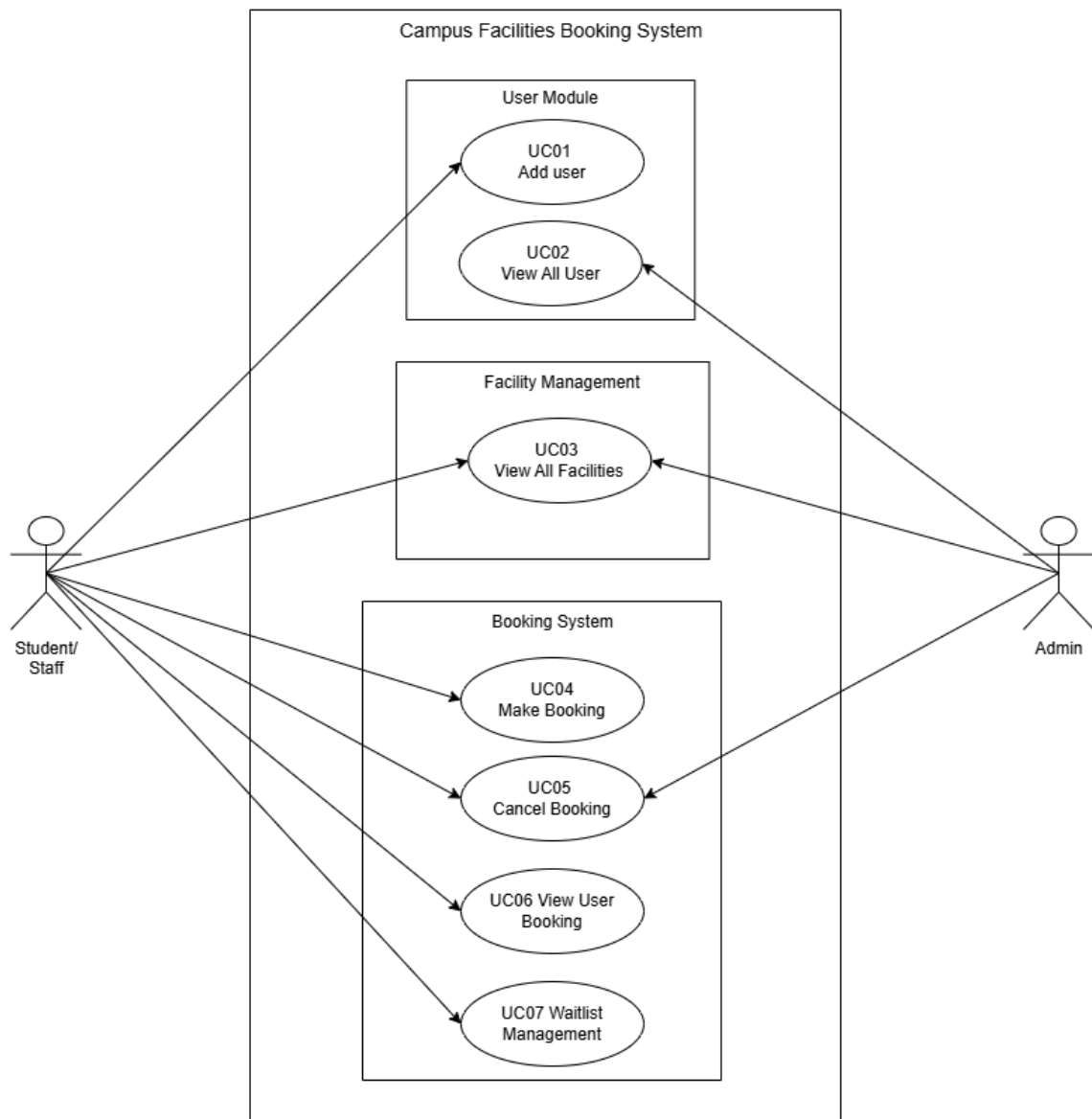


Figure 2.1 Use case diagram of Campus Facilities Booking System

3.0 Class Diagram

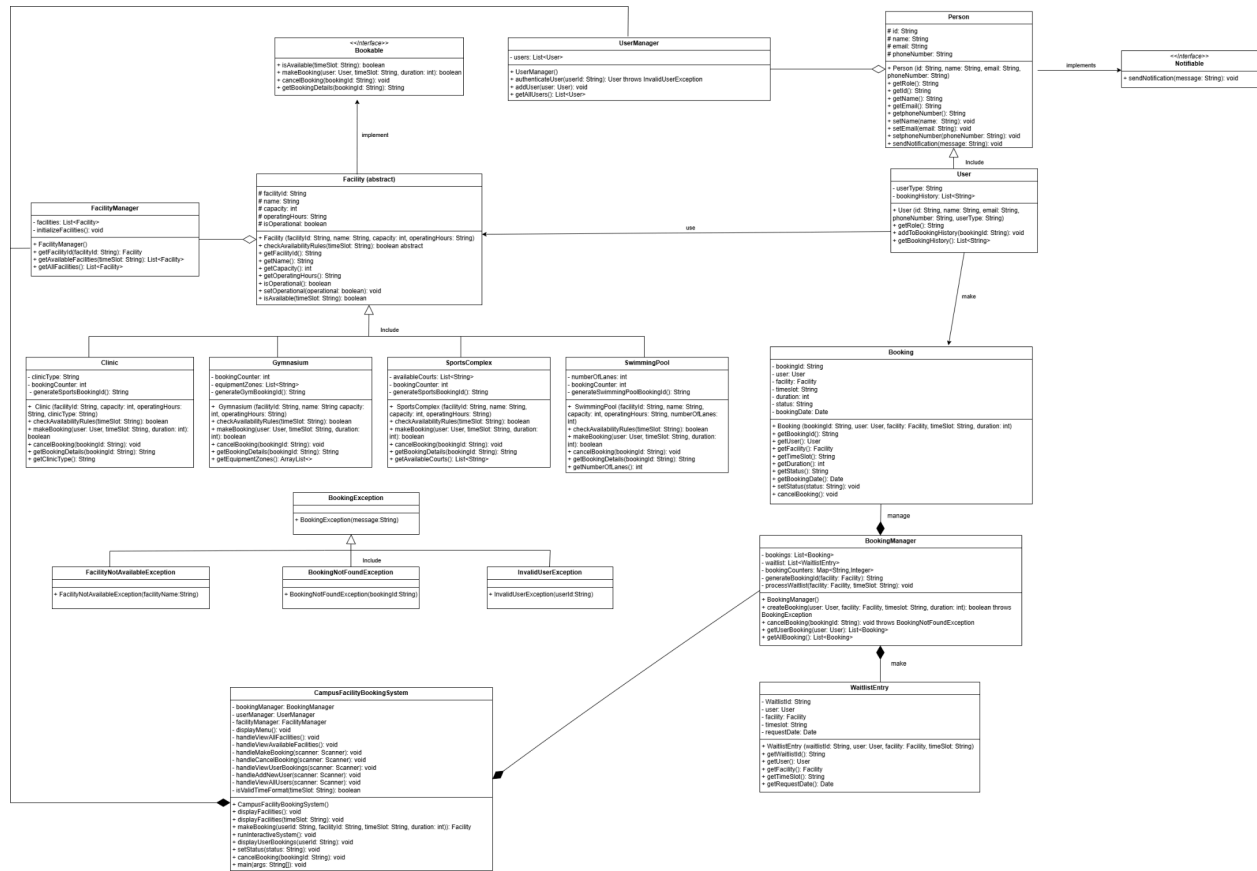


Figure 3.1 Class diagram of Campus Facilities Booking System

4.0 A complete Java program

4.1 Bookable.java

```
public interface Bookable {  
    boolean isAvailable(String timeSlot);  
    boolean makeBooking(User user, String timeSlot, int duration);  
    void cancelBooking(String bookingId);  
    String getBookingDetails(String bookingId);  
}
```

4.2 Facility.java

```
public abstract class Facility implements Bookable {  
    protected String facilityId;  
    protected String name;  
    protected int capacity;  
    protected String operatingHours;  
    protected boolean isOperational;  
  
    public Facility(String facilityId, String name, int capacity, String operatingHours) {  
        this.facilityId = facilityId;  
        this.name = name;  
        this.capacity = capacity;  
        this.operatingHours = operatingHours;  
        this.isOperational = true;  
    }  
  
    // Abstract method for facility-specific rules  
    public abstract boolean checkAvailabilityRules(String timeSlot);  
  
    // Getters (Encapsulation)  
    public String getFacilityId() { return facilityId; }  
    public String getName() { return name; }  
    public int getCapacity() { return capacity; }  
    public String getOperatingHours() { return operatingHours; }  
    public boolean isOperational() { return isOperational; }  
  
    public void setOperational(boolean operational) { this.isOperational = operational; }  
  
    @Override  
    public boolean isAvailable(String timeSlot) {  
        return isOperational && checkAvailabilityRules(timeSlot);  
    }  
}
```

4.3 FacilityManager.java

```
import java.util.*;

public class FacilityManager {
    private final List<Facility> facilities; // Aggregation - FacilityManager uses Facilities

    public FacilityManager() {
        this.facilities = new ArrayList<>();
        initializeFacilities();
    }

    private void initializeFacilities() {
        facilities.add(new Gymnasium("GYM001", "Gym", 30, "08:00-22:00"));
        facilities.add(new SwimmingPool("POOL001", "Swimming Pool", 30, "09:00-17:00", 8));
        facilities.add(new SportsComplex("SPORTS001", "Sports Hall", 50, "08:00-21:00"));
        facilities.add(new Clinic("CLINICM", "PKU (Medical)", 20, "00:00-23:59", "General"));
        facilities.add(new Clinic("CLINICD", "PKU (Dental)", 3, "09:00-17:00", "Dental"));
    }

    public Facility getFacilityById(String facilityId) {
        for (Facility facility : facilities) {
            if (facility.getFacilityId().equals(facilityId)) {
                return facility;
            }
        }
        return null;
    }

    public List<Facility> getAllFacilities() {
        return new ArrayList<>(facilities); // Return copy to maintain encapsulation
    }

    public List<Facility> getAvailableFacilities(String timeSlot) {
        List<Facility> available = new ArrayList<>();
        for (Facility facility : facilities) {
            if (facility.isAvailable(timeSlot)) {
                available.add(facility);
            }
        }
        return available;
    }
}
```


4.4 Gymnasium.java

```
import java.util.*;

public class Gymnasium extends Facility {
    private static int bookingCounter = 1;

    private final List<String> equipmentZones;

    public Gymnasium(String facilityId, String name, int capacity, String operatingHours) {
        super(facilityId, name, capacity, operatingHours);
        this.equipmentZones = new java.util.ArrayList<>();
        equipmentZones.add("Cardio Zone");
        equipmentZones.add("Weight Training Zone");
        equipmentZones.add("Functional Training Zone");
    }

    @Override
    public boolean checkAvailabilityRules(String timeSlot) {
        // Specific rules for gymnasium
        int hour = Integer.parseInt(timeSlot.split(":")[0]);
        return hour >= 6 && hour <= 22; // Open 6 AM to 10 PM
    }

    @Override
    public boolean makeBooking(User user, String timeSlot, int duration) {
        if (isAvailable(timeSlot)) {
            String bookingId = generateGymBookingId();
            user.addToBookingHistory(bookingId);
            System.out.println("Gymnasium booked successfully. Booking ID: " + bookingId);
            return true;
        }
        return false;
    }

    private static synchronized String generateGymBookingId() {
        String formattedNumber = String.format("%05d", bookingCounter);
        bookingCounter++;
        return "GYM" + formattedNumber;
    }

    @Override
    public void cancelBooking(String bookingId) {
        System.out.println("Booking " + bookingId + " cancelled for " + name);
    }

    @Override
    public String getBookingDetails(String bookingId) {
        return "Gymnasium booking details for " + bookingId;
    }

    public List<String> getEquipmentZones() {
        return new ArrayList<>(equipmentZones);
    }
}
```

4.5 SwimmingPool.java

```
public class SwimmingPool extends Facility {
    private final int numberOfLanes;
    private static int bookingCounter = 1;

    public SwimmingPool(String facilityId, String name, int capacity, String operatingHours, int numberOfLanes) {
        super(facilityId, name, capacity, operatingHours);
        this.numberOfLanes = numberOfLanes;
    }

    @Override
    public boolean checkAvailabilityRules(String timeSlot) {
        // Specific rules for swimming pool
        int hour = Integer.parseInt(timeSlot.split(":")[0]);
        return hour >= 8 && hour <= 17; // Open 7 AM to 9 PM
    }

    @Override
    public boolean makeBooking(User user, String timeSlot, int duration) {
        if (isAvailable(timeSlot)) {
            String bookingId = generateSwimmingPoolBookingId();
            user.addToBookingHistory(bookingId);
            System.out.println("Swimming pool booked successfully. Booking ID: " + bookingId);
            return true;
        }
        return false;
    }

    private static synchronized String generateSwimmingPoolBookingId() {
        String formattedNumber = String.format("%05d", bookingCounter);
        bookingCounter++;
        return "POOL" + formattedNumber;
    }

    @Override
    public void cancelBooking(String bookingId) {
        System.out.println("Booking " + bookingId + " cancelled for " + name);
    }

    @Override
    public String getBookingDetails(String bookingId) {
        return "Swimming pool booking details for " + bookingId + " - Lanes available: " + numberOfLanes;
    }

    public int getNumberOfLanes() {
        return numberOfLanes;
    }
}
```

4.6 SportsComplex.java

```
import java.util.*;

public class SportsComplex extends Facility {
    private final List<String> availableCourts;
    private static int bookingCounter = 1; // Facility-specific counter

    public SportsComplex(String facilityId, String name, int capacity, String operatingHours) {
        super(facilityId, name, capacity, operatingHours);
        this.availableCourts = new ArrayList<>();
        availableCourts.add("Basketball Court");
        availableCourts.add("Badminton Court");
        availableCourts.add("Tennis Court");
        availableCourts.add("Squash Court");
    }

    @Override
    public boolean checkAvailabilityRules(String timeSlot) {
        int hour = Integer.parseInt(timeSlot.split(":")[0]);
        return hour >= 8 && hour <= 20; // Open 8 AM to 8 PM
    }

    @Override
    public boolean makeBooking(User user, String timeSlot, int duration) {
        if (isAvailable(timeSlot)) {
            String bookingId = generateSportsBookingId();
            user.addToBookingHistory(bookingId);
            System.out.println("Sports complex booked successfully. Booking ID: " + bookingId);
            return true;
        }
        return false;
    }

    private static synchronized String generateSportsBookingId() {
        String formattedNumber = String.format("%05d", bookingCounter);
        bookingCounter++;
        return "SPORTS" + formattedNumber;
    }

    @Override
    public void cancelBooking(String bookingId) {
        System.out.println("Booking " + bookingId + " cancelled for " + name);
    }

    @Override
    public String getBookingDetails(String bookingId) {
        return "Sports complex booking details for " + bookingId;
    }

    public List<String> getAvailableCourts() {
        return new ArrayList<>(availableCourts);
    }
}
```

4.7 Clinic.java

```
public class Clinic extends Facility {
    private final String clinicType;
    private static int bookingCounter = 1; // Static counter for booking IDs

    public Clinic(String facilityId, String name, int capacity, String operatingHours, String clinicType) {
        super(facilityId, name, capacity, operatingHours);
        this.clinicType = clinicType;
    }

    @Override
    public boolean checkAvailabilityRules(String timeSlot) {
        int hour = Integer.parseInt(timeSlot.split(":")[0]);
        return hour >= 9 && hour <= 17; // Open 9 AM to 5 PM
    }

    @Override
    public boolean makeBooking(User user, String timeSlot, int duration) {
        if (isAvailable(timeSlot)) {
            String bookingId = generateClinicBookingId();
            user.addToBookingHistory(bookingId);
            System.out.println("Clinic appointment booked successfully. Booking ID: " + bookingId);
            return true;
        }
        return false;
    }

    private static synchronized String generateClinicBookingId() {
        String formattedNumber = String.format("%05d", bookingCounter);
        bookingCounter++;
        return "CLINIC" + formattedNumber;
    }

    @Override
    public void cancelBooking(String bookingId) {
        System.out.println("Appointment " + bookingId + " cancelled for " + name);
    }

    @Override
    public String getBookingDetails(String bookingId) {
        return "Clinic appointment details for " + bookingId + " - Type: " + clinicType;
    }

    public String getClinicType() {
        return clinicType;
    }
}
```

4.8 Notifiable.java

```
public interface Notifiable {
    void sendNotification(String message);
}
```

4.9 Person.java

```
public abstract class Person implements Notifiable {
    protected String id;
    protected String name;
    protected String email;
    protected String phoneNumber;

    public Person(String id, String name, String email, String phoneNumber) {
        this.id = id;
        this.name = name;
        this.email = email;
        this.phoneNumber = phoneNumber;
    }

    // Abstract method to be implemented by subclasses
    public abstract String getRole();

    // Getters and setters (Encapsulation)
    public String getId() { return id; }
    public String getName() { return name; }
    public String getEmail() { return email; }
    public String getPhoneNumber() { return phoneNumber; }

    public void setName(String name) { this.name = name; }
    public void setEmail(String email) { this.email = email; }
    public void setPhoneNumber(String phoneNumber) { this.phoneNumber = phoneNumber; }

    @Override
    public void sendNotification(String message) {
        System.out.println("Notification to " + name + " (" + email + "): " + message);
    }
}
```

4.10 User.java

```
import java.util.*;

public class User extends Person {
    private final String userType; // "STUDENT" or "STAFF"
    private final List<String> bookingHistory;

    public User(String id, String name, String email, String phoneNumber, String userType) {
        super(id, name, email, phoneNumber);
        this.userType = userType;
        this.bookingHistory = new ArrayList<>();
    }

    @Override
    public String getRole() {
        return userType;
    }

    public void addToBookingHistory(String bookingId) {
        bookingHistory.add(bookingId);
    }

    public List<String> getBookingHistory() {
        return new ArrayList<>(bookingHistory); // Return copy to maintain encapsulation
    }
}
```

4.11 UserManager.java

```
import java.util.*;

public class UserManager {
    private final List<User> users; // Aggregation - UserManager uses Users

    public UserManager() {
        this.users = new ArrayList<>();
    }

    public User authenticateUser(String userId) throws InvalidUserException {
        for (User user : users) {
            if (user.getId().equals(userId)) {
                return user;
            }
        }
        throw new InvalidUserException(userId);
    }

    public void addUser(User user) {
        users.add(user);
    }

    public List<User> getAllUsers() {
        return new ArrayList<>(users); // Return copy to maintain encapsulation
    }
}
```

4.12 WaitlistEntry.java

```
import java.util.*;

public class WaitlistEntry {
    private final String waitlistId;
    private final User user; // Association with User
    private final Facility facility; // Association with Facility
    private final String timeSlot;
    private final Date requestDate;

    public WaitlistEntry(String waitlistId, User user, Facility facility, String timeSlot) {
        this.waitlistId = waitlistId;
        this.user = user;
        this.facility = facility;
        this.timeSlot = timeSlot;
        this.requestDate = new Date();
    }

    // Getters (Encapsulation)
    public String getWaitlistId() { return waitlistId; }
    public User getUser() { return user; }
    public Facility getFacility() { return facility; }
    public String getTimeSlot() { return timeSlot; }
    public java.util.Date getRequestDate() { return requestDate; }
}
```

4.13 Booking.java

```
import java.util.*;

public class Booking {
    private final String bookingId;
    private final User user; // Association with User
    private final Facility facility; // Association with Facility
    private final String timeSlot;
    private final int duration;
    private String status;
    private final Date bookingDate;

    public Booking(String bookingId, User user, Facility facility, String timeSlot, int duration) {
        this.bookingId = bookingId;
        this.user = user;
        this.facility = facility;
        this.timeSlot = timeSlot;
        this.duration = duration;
        this.status = "CONFIRMED";
        this.bookingDate = new Date();
    }

    // Getters and setters (Encapsulation)
    public String getBookingId() { return bookingId; }
    public User getUser() { return user; }
    public Facility getFacility() { return facility; }
    public String getTimeSlot() { return timeSlot; }
    public int getDuration() { return duration; }
    public String getStatus() { return status; }
    public java.util.Date getBookingDate() { return bookingDate; }

    public void setStatus(String status) { this.status = status; }

    public void cancelBooking() {
        this.status = "CANCELLED";
        facility.cancelBooking(bookingId);
    }
}
```


4.14 BookingManager.java

```
import java.util.*;

public class BookingManager {

    private final List<Booking> bookings; // Composition - BookingManager contains Bookings
    private final List<WaitlistEntry> waitlist; // Composition - BookingManager contains WaitlistEntries

    private final static Map<String, Integer> bookingCounters = new HashMap<>();

    public BookingManager() {
        this.bookings = new ArrayList<>();
        this.waitlist = new ArrayList<>();
    }

    private static synchronized String generateBookingId(Facility facility) {
        String facilityPrefix = "";
        if (facility instanceof Gymnasium) {
            facilityPrefix = "GYM";
        } else if (facility instanceof SwimmingPool) {
            facilityPrefix = "POOL";
        } else if (facility instanceof SportsComplex) {
            facilityPrefix = "SPORTS";
        } else if (facility instanceof Clinic) {
            facilityPrefix = "CLINIC";
        }

        int currentCount = bookingCounters.getOrDefault(facilityPrefix, 0) + 1;
        bookingCounters.put(facilityPrefix, currentCount);
        return facilityPrefix + String.format("%05d", currentCount);
    }

    public boolean createBooking(User user, Facility facility, String timeSlot, int duration) throws BookingException {
        if (user == null) {
            throw new InvalidUserException("null");
        }

        if (!facility.isAvailable(timeSlot)) {
            // Add to waitlist
            String waitlistId = "WAIT-" + System.currentTimeMillis();
            WaitlistEntry entry = new WaitlistEntry(waitlistId, user, facility, timeSlot);
            waitlist.add(entry);
            user.sendNotification("Added to waitlist for " + facility.getName() + " at " + timeSlot);
            throw new FacilityNotAvailableException(facility.getName());
        }

        String bookingId = generateBookingId(facility);

        Booking booking = new Booking(bookingId, user, facility, timeSlot, duration);
        bookings.add(booking);

        // Make the actual booking
        boolean success = facility.makeBooking(user, timeSlot, duration);
        if (success) {
            user.sendNotification("Booking confirmed for " + facility.getName() + " at " + timeSlot);
            return true;
        }
        return false;
    }
}
```

```

public void cancelBooking(String bookingId) throws BookingNotFoundException {
    Booking bookingToCancel = null;
    for (Booking booking : bookings) {
        if (booking.getBookingId().equals(bookingId)) {
            bookingToCancel = booking;
            break;
        }
    }

    if (bookingToCancel == null) {
        throw new BookingNotFoundException(bookingId);
    }

    bookingToCancel.cancelBooking();
    bookingToCancel.getUser().sendNotification("Your booking " + bookingId + " has been cancelled.");

    // Check waitlist for this facility and time slot
    processWaitlist(bookingToCancel.getFacility(), bookingToCancel.getTimeSlot());
}

private void processWaitlist(Facility facility, String timeSlot) {
    for (int i = 0; i < waitlist.size(); i++) {
        WaitlistEntry entry = waitlist.get(i);
        if (entry.getFacility().equals(facility) && entry.getTimeSlot().equals(timeSlot)) {
            User waitlistUser = entry.getUser();
            waitlistUser.sendNotification("A slot is now available for " + facility.getName() + " at " + timeSlot +
                                           ". Reply to confirm booking within 30 minutes.");
            waitlist.remove(i);
            break;
        }
    }
}

public List<Booking> getUserBookings(User user) {
    List<Booking> userBookings = new ArrayList<>();
    for (Booking booking : bookings) {
        if (booking.getUser().equals(user)) {
            userBookings.add(booking);
        }
    }
    return userBookings;
}

public List<Booking> getAllBookings() {
    return new ArrayList<>(bookings); // Return copy to maintain encapsulation
}
}

```

4.15 BookingException.java

```

public class BookingException extends Exception {
    public BookingException(String message) {
        super(message);
    }
}

```

4.16 FacilityNotAvailableException.java

```

public class FacilityNotAvailableException extends BookingException {
    public FacilityNotAvailableException(String facilityName) {
        super("Facility " + facilityName + " is not available at the requested time.");
    }
}

```

4.17 InvalidUserException.java

```
public class InvalidUserException extends BookingException {  
    public InvalidUserException(String userId) {  
        super("User with ID " + userId + " is not valid or not found.");  
    }  
}
```

4.18 BookingNotFoundException.java

```
public class BookingNotFoundException extends Exception {  
    public BookingNotFoundException(String bookingId) {  
        super("Booking with ID " + bookingId + " was not found.");  
    }  
}
```

4.19 CampusFacilityBookingSystem.java (main)

```
import java.util.*;

class CampusFacilityBookingSystem {
    private final BookingManager bookingManager; // Composition
    private final FacilityManager facilityManager; // Composition
    private final UserManager userManager; // Composition

    public CampusFacilityBookingSystem() {
        this.bookingManager = new BookingManager();
        this.facilityManager = new FacilityManager();
        this.userManager = new UserManager();
    }

    // Polymorphism - method overloading
    public void displayFacilities() {
        System.out.println("\n=== All Facilities ===");
        for (Facility facility : facilityManager.getAllFacilities()) {
            System.out.println(facility.getName() + " (" + facility.getFacilityId() + ") - Capacity: " +
                facility.getCapacity() + ", Hours: " + facility.getOperatingHours());
        }
    }

    public void displayFacilities(String timeSlot) {
        System.out.println("\n=== Available Facilities at " + timeSlot + " ===");
        for (Facility facility : facilityManager.getAvailableFacilities(timeSlot)) {
            System.out.println(facility.getName() + " (" + facility.getFacilityId() + ") - Available");
        }
    }

    public boolean makeBooking(String userId, String facilityId, String timeSlot, int duration) {
        try {
            User user = userManager.authenticateUser(userId);
            Facility facility = facilityManager.getFacilityById(facilityId);

            if (facility == null) {
                System.out.println("Facility not found: " + facilityId);
                return false;
            }

            return bookingManager.createBooking(user, facility, timeSlot, duration);
        } catch (InvalidUserException e) {
            System.out.println("User error: " + e.getMessage());
            return false;
        } catch (BookingException e) {
            System.out.println("Booking error: " + e.getMessage());
            return false;
        }
    }

    public void cancelBooking(String bookingId) {
        try {
            bookingManager.cancelBooking(bookingId);
            System.out.println("BOOKING CANCELLED SUCCESSFULLY!!");
        } catch (BookingNotFoundException e) {
            System.out.println("Cancellation error: " + e.getMessage());
        }
    }
}
```

```

public void displayUserBookings(String userId) {
    try {
        User user = userManager.authenticateUser(userId);
        List<Booking> userBookings = bookingManager.getUserBookings(user);

        System.out.println("\n=== Bookings for " + user.getName() + " ===");
        if (userBookings.isEmpty()) {
            System.out.println("NO BOOKING FOUND.");
        } else {
            for (Booking booking : userBookings) {
                if (booking.getStatus().equals("CONFIRMED")) {
                    System.out.println("Booking ID: " + booking.getBookingId() +
                        ", Facility: " + booking.getFacility().getName() +
                        ", Time: " + booking.getTimeSlot() +
                        ", Status: " + booking.getStatus());
                }
            }
        }
    } catch (InvalidUserException e) {
        System.out.println("User error: " + e.getMessage());
    }
}

public void runInteractiveSystem() {
    Scanner scanner = new Scanner(System.in);
    boolean running = true;

    System.out.println("\n\n=== WELCOME TO CAMPUS FACILITY BOOKING SYSTEM ===");

    while (running) {
        displayMenu();
        System.out.print("Choose an option: ");

        try {
            int choice = scanner.nextInt();
            scanner.nextLine(); // Consume newline

            switch (choice) {
                case 1:
                    handleAddNewUser(scanner);
                    break;
                case 2:
                    handleViewAllUsers();
                    break;
                case 3:
                    handleViewAllFacilities();
                    break;
                case 4:
                    handleMakeBooking(scanner);
                    break;
                case 5:
                    handleCancelBooking(scanner);
                    break;
                case 6:
                    handleViewUserBookings(scanner);
                    break;
                case 7:
                    System.out.println("\nThank you for using Campus Facility Booking System!");
                    running = false;
            }
        }
    }
}

```

```

        break;
    default:
        System.out.println("Invalid option. Please try again.");
    }
} catch (InputMismatchException e) {
    System.out.println("Invalid input. Please enter a number.");
    scanner.nextLine(); // Clear invalid input
}
}

scanner.close();
}

private void displayMenu() {
    System.out.println("\n" + "=".repeat(50));
    System.out.println("        CAMPUS BOOKING SYSTEM MENU");
    System.out.println("=".repeat(50));
    System.out.println("1. Add New User");
    System.out.println("2. View All Users");
    System.out.println("3. View All Facilities");
    System.out.println("4. Make a Booking");
    System.out.println("5. Cancel a Booking");
    System.out.println("6. View User Bookings");
    System.out.println("7. Exit");
    System.out.println("=".repeat(50));
}

private void handleViewAllFacilities() {
    displayFacilities();
}

private void handleMakeBooking(Scanner scanner) {
    System.out.println("\n=== Make a New Booking ===");

    // Show available users first
    System.out.println("Available Users:");
    for (User user : userManager.getAllUsers()) {
        System.out.println("- " + user.getId() + " (" + user.getName() + " - " + user.getRole() + ")");
    }

    System.out.print("Enter your Matric No. / Staff ID : ");
    String userId = scanner.nextLine();

    // Authenticate user immediately
    try {
        userManager.authenticateUser(userId);
    } catch (InvalidUserException e) {
        System.out.println("Invalid user: " + e.getMessage());
        return; // Stop here and don't proceed if invalid
    }

    // Show available facilities
    System.out.println("\nAvailable Facilities:");
    for (Facility facility : facilityManager.getAllFacilities()) {
        System.out.println("- " + facility.getFacilityId() + " (" + facility.getName() +
            ") - Capacity: " + facility.getCapacity() +
            ", Hours: " + facility.getOperatingHours());
    }
}

```

```

System.out.print("Enter Facility ID: ");
String facilityId = scanner.nextLine();

System.out.print("Enter time slot (HH:MM format): ");
String timeSlot = scanner.nextLine();

if (!isValidTimeFormat(timeSlot)) {
    System.out.println("Invalid time format. Please use HH:MM format.");
    return;
}

System.out.print("Enter duration in hours: ");
try {
    int duration = scanner.nextInt();
    scanner.nextLine(); // Consume newline

    boolean success = makeBooking(userId, facilityId, timeSlot, duration);
    if (success) {
        System.out.println("BOOKING SUCCESSFULLY!");
    } else {
        System.out.println("Booking failed. Please check the details and try again.");
    }
} catch (InputMismatchException e) {
    System.out.println("Invalid duration. Please enter a number.");
    scanner.nextLine(); // Clear invalid input
}
}

private void handleCancelBooking(Scanner scanner) {
    System.out.println("\n=== Cancel a Booking ===");
    System.out.print("Enter Booking ID to cancel: ");
    String bookingId = scanner.nextLine();

    if (bookingId.trim().isEmpty()) {
        System.out.println("Booking ID cannot be empty.");
        return;
    }

    cancelBooking(bookingId);
}

private void handleViewUserBookings(Scanner scanner) {
    System.out.println("\n=== View User Bookings ===");

    // Show available users
    System.out.println("Available Users:");
    for (User user : userManager.getAllUsers()) {
        System.out.println("- " + user.getId() + " (" + user.getName() + ")");
    }

    System.out.print("Enter Matric No. / Staff ID : ");
    String userId = scanner.nextLine();

    displayUserBookings(userId);
}

```

```

private void handleAddNewUser(Scanner scanner) {
    System.out.println("\n=== Add New User ===");

    System.out.print("Enter Matric No. / Staff ID : ");
    String userId = scanner.nextLine();

    System.out.print("Enter Name: ");
    String name = scanner.nextLine();

    System.out.print("Enter Email: ");
    String email = scanner.nextLine();

    System.out.print("Enter Phone Number: ");
    String phone = scanner.nextLine();

    System.out.print("Enter User Type (STUDENT/STAFF): ");
    String userType = scanner.nextLine().toUpperCase();

    if (!userType.equals("STUDENT") && !userType.equals("STAFF")) {
        System.out.println("Invalid user type. Please enter STUDENT or STAFF.");
        return;
    }

    // Check if user already exists
    try {
        userManager.authenticateUser(userId);
        System.out.println("User with Matric No. / Staff ID " + userId + " already exists.");
        return;
    } catch (InvalidUserException e) {
        // User doesn't exist, we can add them
    }

    User newUser = new User(userId, name, email, phone, userType);
    userManager.addUser(newUser);
    System.out.println("\nUSER ADDED SUCCESSFULLY!\n");
    System.out.println("User Details: " + name + " (" + userId + ") - " + userType);
}

private void handleViewAllUsers() {
    System.out.println("\n=== All Registered Users ===");
    java.util.List<User> users = userManager.getAllUsers();

    if (users.isEmpty()) {
        System.out.println("No users registered.");
    } else {
        int count = 1;
        for (User user : users) {
            System.out.println(count + ". Matric No. / Staff ID: " + user.getId() +
                ", Name: " + user.getName() +
                ", Email: " + user.getEmail() +
                ", Type: " + user.getRole() +
                ", Phone: " + user.getPhoneNumber());

            count++;
        }
    }
}

```



```

private boolean isValidTimeFormat(String timeSlot) {
    if (timeSlot == null || timeSlot.trim().isEmpty()) {
        return false;
    }

    String[] parts = timeSlot.split(":");
    if (parts.length != 2) {
        return false;
    }

    try {
        int hour = Integer.parseInt(parts[0]);
        int minute = Integer.parseInt(parts[1]);
        return hour >= 0 && hour <= 23 && minute >= 0 && minute <= 59;
    } catch (NumberFormatException e) {
        return false;
    }
}

public static void main(String[] args) {
    CampusFacilityBookingSystem system = new CampusFacilityBookingSystem();

    // Run the interactive system
    system.runInteractiveSystem();
}
}

```

5.0 Task Allocation for each member of the group

Tan Zhao Hong	Coding in Main system, class diagram
Chin Pei Wen	Coding in User module, introduction
Koo Xuan	Coding in Booking module, conclusion
Nazatul Nadhirah	Coding in Facility module, use case diagram

6.0 Explanation of the object-oriented concepts applied

The Campus Facility Booking System is structured using fundamental object-oriented programming (OOP) concepts to create a modular, maintainable, and extensible application. The main OOP principles applied are Encapsulation, Inheritance, Polymorphism, and Abstraction, supported by the use of Interfaces and Exception Handling to ensure robustness and flexibility.

6.1 Encapsulation and Data Hiding

Encapsulation is applied by combining related attributes and methods into classes, while restricting direct access to internal data using access modifiers such as private and protected.

For instance, in the User and Facility classes, fields such as user details or facility capacity are made private. Public getter and setter methods are used to control access and modification of these fields. This design ensures that sensitive or critical data is protected from unintended changes, supporting system integrity and secure data handling.

```
private final String bookingId;
private final User user; // Association with User
private final Facility facility; // Association with Facility
private final String timeSlot;
private final int duration;
private String status;
private final java.util.Date bookingDate;
```

Booking implemented encapsulation through private fields

```
public String getBookingId() { return bookingId; }
public User getUser() { return user; }
public Facility getFacility() { return facility; }
public String getTimeSlot() { return timeSlot; }
public int getDuration() { return duration; }
public String getStatus() { return status; }
public java.util.Date getBookingDate() { return bookingDate; }

public void setStatus(String status) { this.status = status; }
```

Booking implemented encapsulation through getters and setter

6.2 Class Relationships

Our system clearly illustrates various types of class relationships:

- Association: The User class is associated with the Booking class, as a user can make multiple bookings.

```
private final String bookingId;  
private final User user; // Association with User  
private final Facility facility; // Association with Facility  
private final String timeSlot;  
private final int duration;  
private String status;  
private final java.util.Date bookingDate;
```

Booking implemented association through the use of User and Facility objects

- Aggregation: The FacilityManager class manages multiple Facility objects. The facilities can exist independently of the manager.

```
private final java.util.List<Facility> facilities; // Aggregation - FacilityManager uses Facilities
```

FacilityManager implemented

- Composition: The BookingManager class owns and controls Booking and WaitlistEntry objects. If the booking manager is destroyed, its contained objects should also be discarded.

```
private final java.util.List<Booking> bookings; // Composition - BookingManager contains Bookings  
private final java.util.List<WaitlistEntry> waitlist; // Composition - BookingManager contains WaitlistEntries
```

BookingManager contains Booking and WaitlistEntry objects

- Inheritance: The class Facility serves as a superclass to Gymnasium, SwimmingPool, SportsComplex, and Clinic, allowing common attributes and methods to be reused. Similarly, the User class extends the Person superclass.

```
public class Clinic extends Facility {  
    private final String clinicType;  
    private static int bookingCounter = 1; // Static counter for booking IDs
```

```
public class Gymnasium extends Facility {  
    private static int bookingCounter = 1;
```

```
public class SportsComplex extends Facility {  
    private final java.util.List<String> availableCourts;  
    private static int bookingCounter = 1; // Facility-specific counter
```

```
public class SwimmingPool extends Facility {  
    private final int numberOfLanes;  
    private static int bookingCounter = 1;
```

These relationships help structure the system in a modular and logical manner, supporting reusability and separation of concerns.

6.3 Polymorphism

Polymorphism enables the system to treat objects of different classes through a common interface or superclass. For example, all subclasses of Facility can be referred to using a Facility reference, allowing generalized operations like `checkAvailabilityRules()`, `makeBook()`, `cancelBoking()` and `getBookingDetails()` to be performed regardless of the specific facility type. Polymorphism provides flexibility and scalability when adding new features or types.

```
@Override
public boolean checkAvailabilityRules(String timeSlot) {
    int hour = Integer.parseInt(timeSlot.split(regex:":")[0]);
    return hour >= 9 && hour <= 17; // Open 9 AM to 5 PM
}

@Override
public boolean makeBooking(User user, String timeSlot, int duration) {
    if (isAvailable(timeSlot)) {
        String bookingId = generateClinicBookingId();
        user.addToBookingHistory(bookingId);
        System.out.println("Clinic appointment booked successfully. Booking ID: " + bookingId);
        return true;
    }
}
```

```
@Override
public void cancelBooking(String bookingId) {
    System.out.println("Appointment " + bookingId + " cancelled for " + name);
}

@Override
public String getBookingDetails(String bookingId) {
    return "Clinic appointment details for " + bookingId + " - Type: " + clinicType;
}
```

The class overrides methods like `makeBooking()`, `cancelBooking()`, `checkAvailabilityRules()`, and `getBookingDetails()`, showing runtime polymorphism

```
// Polymorphism - method overloading
public void displayFacilities() {
    System.out.println(x:"\n=== All Facilities ===");
    for (Facility facility : facilityManager.getAllFacilities()) {
        System.out.println(facility.getName() + " (" + facility.getFacilityId() + ") - Capacity: " +
            facility.getCapacity() + ", Hours: " + facility.getOperatingHours());
    }
}

public void displayFacilities(String timeSlot) {
    System.out.println("\n=== Available Facilities at " + timeSlot + " ===");
    for (Facility facility : facilityManager.getAvailableFacilities(timeSlot)) {
        System.out.println(facility.getName() + " (" + facility.getFacilityId() + ") - Available");
    }
}
```

Method Overloading is shown with `displayFacilities()` and `displayFacilities(String timeSlot)` — compile-time polymorphism

6.4 Interfaces and Implementation

Interfaces are used to define common capabilities across unrelated classes.

- The Bookable interface is implemented by all facilities that can be reserved. This ensures consistency in how facilities handle bookings.

```
public interface Bookable {  
    boolean isAvailable(String timeSlot);  
    boolean makeBooking(User user, String timeSlot, int duration);  
    void cancelBooking(String bookingId);  
    String getBookingDetails(String bookingId);  
}
```

- The Notifiable interface provides a contract for sending notifications, allowing the system to notify users in a uniform way.

```
public interface Notifiable {  
    void sendNotification(String message);  
}
```

Interfaces allow different parts of the system to interoperate more easily and support future extensions.

6.5 Exception Handling

Custom exceptions are used to handle errors gracefully and inform users when something goes wrong. Examples include `BookingException`, `FacilityNotAvailableException`, `InvalidUserException`, and `BookingNotFoundException`. These are thrown and caught throughout the system to manage issues such as double bookings, invalid input, or unavailable facilities. This improves the system's stability, prevents crashes, and enhances user experience by providing clear error messages.

```
public class BookingException extends Exception {  
    public BookingException(String message) {  
        super(message);  
    }  
}
```

```
public class BookingNotFoundException extends Exception {  
    public BookingNotFoundException(String bookingId) {  
        super("Booking with ID " + bookingId + " was not found.");  
    }  
}
```

```
public class FacilityNotAvailableException extends BookingException {  
    public FacilityNotAvailableException(String facilityName) {  
        super("Facility " + facilityName + " is not available at the requested time.");  
    }  
}
```

```
public class InvalidUserException extends BookingException {  
    public InvalidUserException(String userId) {  
        super("User with Matric No. / Staff ID " + userId + " is not valid or not found.");  
    }  
}
```

7.0 Conclusion

The Campus Facility Booking System successfully addresses the core issue of managing campus resource reservations through a structured, efficient, and user-friendly platform. This Java-based application not only streamlines the booking process for students and staff but also enhances transparency and fairness in resource allocation across various campus facilities such as gymnasiums, courts, clinics, and swimming pools.

Throughout the development process, our group effectively applied object-oriented programming principles, including encapsulation, inheritance, polymorphism, interface implementation, and exception handling. These concepts were integrated into different parts of the system. For example, the use of inheritance between the Person and User classes, the implementation of the Bookable and Notifiable interfaces for facility management and notifications, and polymorphism through overridden methods in facility subclasses.

Collaboration among group members allowed us to divide and conquer different modules efficiently, ensuring that each feature such as booking, user, facility tracking, and notification handling, was developed and tested systematically. This project has not only improved our technical proficiency in Java but also strengthened our understanding of how software can solve real-world organizational problems.

Overall, the project reinforces the importance of modular design and teamwork in software engineering while delivering a practical solution that promotes a healthy and active campus lifestyle.